

AI FAQ Assistant API

5. Project Development Phase

1. Introduction

This document describes how the AI FAQ Assistant API was implemented, based on the architecture defined in the Design phase and the schedule defined in the Planning phase. It covers the development approach, technology stack, module-by-module implementation details, and coding practices followed by the team.

2. Development Approach

An iterative, Agile-inspired approach was followed, with development broken into short, focused sprints rather than attempting to build the entire system at once. This allowed the team to test and validate each part of the system before building on top of it.

Sprint	Focus
Sprint 1	Project setup, repository structure, database schema creation
Sprint 2	Core /api/ask endpoint and NLP matching engine integration
Sprint 3	FAQ management (CRUD) endpoints and category support
Sprint 4	Authentication and admin route protection
Sprint 5	Query logging, feedback capture, and basic analytics endpoints

3. Technology Stack

Layer	Technology Used
Backend framework	Node.js (Express) or Python (FastAPI/Flask)
Database	MySQL / PostgreSQL
NLP / semantic matching	Sentence-embedding model or AI API for similarity scoring

Authentication	JSON Web Tokens (JWT)
API documentation	Swagger / OpenAPI specification
Version control	Git, hosted on GitHub

4. Environment Setup

- Initialized a version-controlled repository with a clear folder structure separating routes, controllers, models, and services
- Configured environment variables for database credentials, API keys, and secret keys (never hard-coded)
- Set up a local development database matching the schema defined in the Design phase
- Configured a base server with health-check endpoint to confirm the environment was working before feature development began

5. Module-wise Development Details

5.1 FAQ Management Module

Implements CRUD (Create, Read, Update, Delete) endpoints for FAQs and categories. Each write operation validates required fields (question, answer, category) before persisting to the database, and all admin-only routes are protected by the authentication middleware.

5.2 NLP Matching Module

At creation or update time, each FAQ's question text is converted into a numeric embedding vector and stored alongside the FAQ record. When a user submits a question through `/api/ask`, the same embedding process is applied to the incoming text, and cosine similarity is calculated against all stored FAQ embeddings to identify the closest matches. Matches above the configured confidence threshold are returned as answers; matches below the threshold trigger the fallback response.

5.3 Query and Response API

The `/api/ask` endpoint orchestrates the full request lifecycle: validating the incoming question, invoking the NLP Matching Module, selecting the best result, formatting the JSON response (answer, confidence score, and related FAQ id), and triggering the logging step.

5.4 Authentication Module

Implements admin login, verifying submitted credentials against the stored password hash and issuing a signed JWT access token on success. A reusable middleware function checks this token on every admin-only request and rejects the request with an appropriate error if the token is missing, invalid, or expired.

5.5 Analytics and Feedback Module

Every call to `/api/ask` results in a new row in the `query_logs` table, capturing the original question, the matched FAQ id (if any), the confidence score, and a timestamp. The feedback endpoint links a helpful/not-helpful rating back to a specific query log entry, enabling later analysis of which answers are actually useful to users.

6. Sample API Request/Response Behaviour

Endpoint	Sample Input	Sample Output
POST <code>/api/ask</code>	<code>{"question": "How do I reset my password?"}</code>	<code>{"answer": "Go to Settings > Security > Reset Password.", "confidence": 0.91}</code>
POST <code>/api/faqs</code>	<code>{"question": "How do I reset my password?", "answer": "...", "category": "Account"}</code>	<code>{"id": 101, "status": "created"}</code>
PUT <code>/api/faqs/101</code>	<code>{"answer": "Updated answer text"}</code>	<code>{"id": 101, "status": "updated"}</code>
POST <code>/api/feedback</code>	<code>{"query_log_id": 55, "is_helpful": true}</code>	<code>{"status": "recorded"}</code>
POST <code>/api/auth/login</code>	<code>{"email": "admin@example.com", "password": "..."} </code>	<code>{"token": "eyJhbGciOi..."}</code>

7. Coding Standards and Practices

- Consistent naming conventions and a modular folder structure (routes / controllers / services / models)

- Environment variables used for all secrets and configuration values — no credentials hard-coded in source
- Input validation applied on every API endpoint before processing
- Centralized error-handling middleware to return consistent, informative error responses
- Meaningful, descriptive Git commit messages and a feature-branch workflow
- Code comments added for non-obvious logic, particularly around the NLP matching threshold

8. Challenges Encountered During Development

Challenge	Resolution
Deciding an appropriate confidence threshold	Tested multiple threshold values against sample questions and adjusted based on observed accuracy
Slow response time when computing embeddings on every request	Pre-computed and cached FAQ embeddings at creation/update time instead of recalculating them for every query
Protecting admin routes without duplicating logic	Implemented a single reusable authentication middleware applied to all admin routes

9. Version Control Practices

All source code was tracked using Git, with feature branches created for each module (e.g., feature/faq-crud, feature/auth). Pull requests were used to review changes before merging into the main branch, helping catch issues early and keeping a clean project history.

10. Pseudocode: Core Matching Logic

The following simplified pseudocode illustrates the core logic used inside the NLP Matching Module to select an answer for an incoming question:

- function findBestMatch(question):
- queryEmbedding = embed(question)

- `scores = []`
- `for each faq in faqDatabase:`
- `score = cosineSimilarity(queryEmbedding, faq.embedding_vector)`
- `scores.append((faq, score))`
- `bestFaq, bestScore = highest score in scores`
- `if bestScore >= CONFIDENCE_THRESHOLD:`
- `return bestFaq.answer, bestScore`
- `else:`
- `return FALLBACK_MESSAGE, bestScore`

11. Integration Notes

The NLP Matching Module was integrated behind a simple internal interface (`findBestMatch`) so that the rest of the API layer does not need to know which specific embedding technique or library is used underneath. This matches the design goal from the Design phase of keeping the matching engine replaceable without impacting other modules.

12. Deployment Preparation During Development

- Configured separate environment variable files for local development and staging deployment
- Added a health-check endpoint (`/api/health`) to confirm the service and database connection are running correctly
- Documented all endpoints using Swagger/OpenAPI as they were completed, rather than leaving documentation until the end

13. Code Review and Merge Process

Every feature branch was reviewed by at least one other team member before being merged into the main branch. Reviews focused on correctness against the relevant functional requirement, adherence to the coding standards in Section 7, and whether adequate error handling was included.

This process helped catch several small issues — such as missing input validation — before they reached the Testing phase.

14. Traceability to Requirements

Requirement	Implemented In
FR-1, FR-2, FR-3, FR-4	Query and Response API, NLP Matching Module (Sections 5.2, 5.3)
FR-5, FR-6, FR-7	FAQ Management Module (Section 5.1)
FR-9	Analytics and Feedback Module — query logging (Section 5.5)
FR-10	Analytics and Feedback Module — feedback capture (Section 5.5)
FR-11	Authentication Module (Section 5.4)

15. Post-Development Review

At the end of the development phase, the team walked through each Must-have requirement against the working system to confirm nothing was missed before formally handing the build over to the Testing phase. Minor polish items (such as improved error message wording) were noted but deferred, as they did not affect core functionality or requirement compliance.

16. Detailed Walkthrough of the FAQ Creation Flow

To illustrate how the modules described earlier work together in practice, this section walks through the full lifecycle of creating a new FAQ entry and having it become answerable through the API.

- 1. An authenticated administrator sends a POST request to `/api/faqs` with a question, answer, and category.
- 2. The `AuthMiddleware` verifies the admin's token before the request reaches the `FaqController`.
- 3. The `FaqController` passes the validated data to the `FaqService`, which checks that required fields are present and well-formed.

- 4. The `FaqService` calls the `MatchingService` to generate an embedding vector for the new question text.
- 5. The `FaqService` persists the new FAQ record — including its embedding vector — into the `faqs` table via the database layer.
- 6. From this point onward, the new FAQ is immediately eligible to be matched against incoming user questions through `/api/ask`, with no separate 'publish' step required.

This walkthrough highlights why pre-computing the embedding at creation time (rather than at query time) was an important development decision: it keeps the `/api/ask` endpoint fast, since no embedding computation is needed for existing FAQs during a live user query.

17. Developer Notes for Future Maintainers

A short set of internal notes was documented alongside the code to help future developers extend the system without needing to reverse-engineer the original design decisions:

- The confidence threshold is stored as a configuration value, not hard-coded, so it can be tuned without a code change and redeployment
- The `MatchingService` interface is intentionally minimal (a single `findBestMatch` function) so that the underlying NLP technique can be replaced later with minimal impact elsewhere
- All timestamps are stored in a consistent format to simplify future analytics and reporting work

18. Conclusion

The development phase successfully implemented all Must-have functional requirements defined earlier: the question-answering API, FAQ management, authentication, and query/feedback logging. The resulting codebase is modular, documented, and ready to proceed into the Testing phase for formal verification against the documented requirements.